SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Naveen Kumar S	**Reg. No.:**	192421030
Section / Batch:	B Tech IT	**Date:**	27th July

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems. • Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required. • Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis. • Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Linear Search – First Element Detection

Given the unsorted dataset [55, 21, 39, 47, 63, 70], apply Linear Search to find the element 55. Clearly interpret the problem and explain the search process step-by-step. Count the number of comparisons required to find the target element. Analyze the time complexity in best, average, and worst cases. Verify the correctness of the result. Interpret the efficiency of Linear Search when the target is found immediately.

Code of Conduct

I Naveen Kumar S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the StudentNaveen Kumar S

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Answer:

Linear Search – First Element Detection

1. Introduction

Linear search examines each element sequentially until the target is found or the list ends. Dataset: [55, 21, 39, 47, 63, 70]. Target: 55.

2. Problem Interpretation

The dataset is unsorted, so linear search is appropriate because it does not require sorted data. The objective is to locate 55 and count comparisons.

3. Step-by-Step Search Process

Step 1: Compare first element (55) with target (55).
Result: Match found immediately.
Search stops.
Comparisons = 1.

4. Verification

The first element equals the target, therefore the algorithm correctly returns index 0 (first position).

5. Time Complexity Analysis

Best case: O(1) – target at first position (this example).
Average case: O(n) – target around the middle.
Worst case: O(n) – target last or absent.

6. Efficiency Interpretation

When the target is found immediately, linear search performs only one comparison, making execution extremely efficient despite its linear worst-case behaviour. No additional elements are examined.

7. Conclusion

The search successfully found 55 after one comparison. The result is correct, and this example demonstrates the best-case performance of linear search.

Discussion: In this example, only the first element is inspected because the target is located immediately. This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.

Discussion: In this example, only the first element is inspected because the target is located immediately. This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.

Discussion: In this example, only the first element is inspected because the target is located immediately. This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.

Discussion: In this example, only the first element is inspected because the target is located immediately. This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.

Discussion: In this example, only the first element is inspected because the target is located immediately. This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.

Discussion: In this example, only the first element is inspected because the target is located immediately. This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.

Discussion: In this example, only the first element is inspected because the target is located immediately. This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.

Discussion: In this example, only the first element is inspected because the target is located immediately.

This highlights that although linear search has O(n) average and worst-case complexity, its best-case performance is constant time O(1). The algorithm is simple, requires no preprocessing, and is suitable for small or unsorted datasets.